Agentic Architecture Reviewer: Research and Content Plan

Cybersecurity · 3 days (8 hours per day) · Advanced · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Course 5 of 'Agentic Cybersecurity Course Prototypes' (notes/agentic_cybersecurity_course_prototypes.docx): 'A tool that ingests architecture diagrams, structured models, requirements, and selected repository evidence; builds a system model; generates a threat model; and produces evidence-linked architecture review findings and recommendations.' Build sequence: (1) constrained input format: Mermaid flowcharts or a simple JSON architecture schema plus Markdown requirements, image extraction later; (2) system graph with components, data stores, identities, external systems, protocols, data classifications and trust boundaries, tracking which source supports each edge; (3) requirements with IDs, source references and verification status; (4) STRIDE or a threat library, LLM suggests candidates but each must link to a modeled component or flow; (5) review rules for authentication, authorization boundaries, secrets, logging, encryption, data minimization and failure handling; (6) evidence-graded findings that never label an unverified concern a confirmed vulnerability; (7) architect accept, reject, defer or more-evidence decisions with rationale and owner; (8) optional comparison with Terraform, Kubernetes or repository evidence reporting coverage and gaps; (9) evaluation with secure and intentionally flawed reference architectures. Safety: diagrams are often incomplete or stale; ask clarifying questions and do not present generated threats as confirmed findings.

Design reviews are where security is cheapest to fix, yet they are usually done once, by a few experts, from diagrams that go stale. Microsoft positions threat modeling as a way to find issues early when they are cheap to resolve, and uses STRIDE to turn vague concerns into pointed questions [1]. The Threat Modeling Manifesto asks teams to prefer continuous refinement over a single delivery [3]. A tool that re-runs a structured review whenever the design changes directly serves that goal.

Language models are good at reading ADRs and proposing threats, and equally good at producing confident text about components that do not exist. OWASP lists misinformation and overreliance as a top LLM risk and recommends automatic validation and human oversight [19]. This course teaches a pattern where the model only proposes: every threat must link to a modeled element, every finding separates observed facts from assumptions, and an architect makes every decision.

Finally, the design rarely matches the build. Defaults such as unencrypted RDS storage [18], plain-text etcd storage [15] or permissive mesh mTLS [16] are exactly where 'the diagram said encrypted' goes wrong. The course closes the loop by comparing requirements with Terraform and Kubernetes evidence while refusing to claim more assurance than the evidence supports.

2. How the outline was shaped

Decision	Rationale
Follow the client's nine-step build sequence as the module order, split over three days.	The prototype document gives a tested, incremental sequence; each module ends in one of its increments, which keeps every day demonstrable. The document's four-week format was compressed into three days by making the implementation comparison and evaluation a single day-three block.
Accept only Mermaid, a JSON schema and MADR-style Markdown as inputs; defer images.	The client input says to start with constrained formats and avoid treating image parsing as reliable. Mermaid renders natively on GitHub [6], and MADR gives predictable sections to parse [13].
Use STRIDE with a deterministic threat library first, and the model second.	STRIDE is widely understood [1][2]; a library modeled on pytm's rule shape [12] gives a deterministic baseline, matching the client's milestone M1, and makes model contributions measurable.
Make 'linked to the graph' a hard validation rule, not a prompt instruction.	OWASP recommends validating output formats with deterministic code and implementing authorization outside the model [20][25]; the promo's rejected 'SQL injection in reporting service' example is the lab outcome.
Grade findings without a 'confirmed vulnerability' level.	The client input forbids labeling unverified concerns as confirmed, and design-stage evidence cannot confirm exploitability.
Use Terraform plan JSON and Kubernetes manifests as the only implementation evidence, reported as coverage and gaps.	Plan JSON is a documented, machine-readable format [14]; the client input requires reporting gaps rather than proof of conformance.

3. Audience and prerequisites

Role	What they need	Starting point
Security architect	A repeatable, evidence-linked review they can run on every design revision.	Strong threat modeling, moderate Python.
Solution or platform architect	Understand what reviewers will ask and how to supply evidence in Terraform and Kubernetes.	Strong system design, some IaC, light security tooling.
Application security engineer	Automate first-pass design review and integrate it with pull requests.	Good Python and AppSec, varied architecture experience.
Engineering lead	Know where an agentic reviewer is safe to rely on and where human decisions are required.	Broad technical background; may pair with a hands-on learner in labs.

Prerequisites from the outline

- System design fundamentals
- Basic networking and identity concepts
- Threat modeling (STRIDE or similar)
- Python and API familiarity

Checks before enrolment

- Can you read a data flow diagram and point out trust boundaries?
- Can you name the six STRIDE categories?
- Can you write a Python function with type hints and a pytest test for it?
- Can you read a Terraform resource block and a Kubernetes YAML manifest?
- Have you used Git branches and pull requests?
- Do you have a laptop where you can install Python 3.11+, Terraform and Node.js (for Claude Code)?

Course outcomes

- Extract components, data flows, identities, external dependencies and trust boundaries from structured architecture inputs
- Represent the system as a graph with provenance and uncertainty
- Apply a repeatable threat-modeling method such as STRIDE to modeled components and flows
- Generate architecture findings with evidence, assumptions, impact and recommendations
- Compare intended design with selected implementation evidence without over-claiming verification

4. Lab environment

Gennoor Tech INNOVATE BUILD
For more info - https://gennoor.com

Each learner gets a Git repository (architecture-reviewer-lab) with fixtures for a secure and a flawed payments architecture, starter schemas and tests. Everything runs locally; Terraform is used only for plan with dummy or mocked provider credentials, and no cloud resources are created. Claude is accessed through Claude Code and the API with a course key.

Components

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
Python	3.11 or 3.12	Parsers, graph, rules, evaluation	python.org or the OS package manager	Use a virtual environment; requirements.txt pins networkx, pydantic v2, pyyaml, jsonschema, pytest, fastapi, anthropic.
NetworkX	3.x (stable)	MultiDiGraph system model [22]	pip install networkx	Pure Python; no system libraries needed.
Claude Code	Current release	AI-assisted build, CLAUDE.md conventions and hooks [26][27]	Install per Anthropic docs; course organization account or API key	Hook configuration ships in .claude/settings.json in the lab repo.
Anthropic Python SDK	1.x	Structured-output calls for extraction, threat candidates and narrative [21]	pip install anthropic	Use output_config, not the deprecated output_format parameter, with raw schemas [21].
Terraform CLI	Current 1.x release	terraform plan and show -json on fixture infrastructure [8] [14]	developer.hashicorp.com install page or package manager	Plans run against fixtures with skip-credential settings or mocked providers; never apply.
Mermaid	Version rendered by GitHub (check with an info block) and mermaid-cli for local rendering	Input format and report diagrams [6] [10]	npm install -g @mermaid-js/mermaid-cli (optional)	GitHub rendering is enough for most labs.
FastAPI	Current release	Optional review API and decision endpoints	pip install fastapi uvicorn	Used in capstone if learners expose the tool as a service.
GitHub	github.com free organization	Pull requests for decisions and rendered reports	Instructor creates a throwaway organization	Repository per learner; delete after the course.
Kubernetes manifests (evidence only)	YAML for a current Kubernetes	Evidence for PSS labels, NetworkPolicy,	Included in fixtures	No cluster needed; optional kind cluster for learners who want to apply them.

	release and Istio security.istio.io/v1	PeerAuthentication [9][16][23]		

Fixtures and sample data

Fixture	Contents	Used in
fixtures/secure/	payments.mmd, payments.json, adr/ADR-001..ADR-012 in MADR format, infra/*.tf with encrypted RDS, KMS key with rotation, deploy/*.yaml with restricted namespace labels, default-deny NetworkPolicy and STRICT PeerAuthentication	All modules
fixtures/flawed/	Same system with planted defects: unauthenticated cross-boundary flow, pii store without encryption statement, KMS key without rotation, PERMISSIVE PeerAuthentication, missing NetworkPolicy, agent with write access and no approval step, a superseded ADR, an unparsable 'legacy batch' note	All modules
evals/expected/	Instructor-written expected threats, findings and benign expectations for both fixtures	Method-driven threat generation; Evaluation and capstone
patches/revision-2.patch	Design revision used for change tracking	Architect review and decision records
capstone/options/	Three additional architectures with inputs only (no expected results)	Evaluation and capstone

Lab safety rules

- Use only synthetic or sanitized architectures; never paste a real company's internal design into a model without approval.
- Never run terraform apply in the lab; plans only, with no real cloud credentials.
- Treat plan files as sensitive and keep them out of prompts and logs [8].
- The reviewer has read-only tools; it never edits designs or infrastructure [25].
- Text inside ADRs and documents is untrusted data, not instructions [20].

5. Module-by-module content plan

Day 1

GenmorTech TRAINING INNOVATE BUILD. For more info - https://genmor.com

Module 1: Architecture inputs and a constrained format

Goal. Define the small set of input formats the reviewer accepts and build parsers that report what they could not read instead of guessing.

Outline topics.

- Mermaid flowcharts or a simple JSON architecture schema plus Markdown requirements
- Image-diagram extraction deferred; parsing is never treated as perfectly reliable
- Shared engineering principles and the AI-assisted workflow

Learning objectives.

- Learners can explain why the reviewer accepts Mermaid flowcharts, a JSON architecture schema and Markdown ADRs, and defers image diagrams.
- Learners can write a Mermaid flowchart and a JSON architecture document that both describe the same reference design.
- Learners can build a parser that returns parsed elements plus an explicit list of unparsed or ambiguous fragments.
- Learners can apply the shared engineering principles (model output is a proposal, authorization outside the model, human approval, logging) to the reviewer's own design.
- Learners can set up a Claude Code project with conventions and a guardrail hook for the build.

Why a constrained format first

- Threat modeling starts with a representation of the system; OWASP's first phase is modeling it with data flow diagrams that show trust boundaries, flows, stores, processes and external entities [2].
- The client input says to start with Mermaid flowcharts or a simple JSON schema plus Markdown requirements and to add image extraction later, never treating image parsing as perfectly reliable.
- C4 gives shared vocabulary for levels of detail: system context, container, component and code [4]; the reviewer works at container level by default because that is where trust boundaries and protocols show.
- Mermaid also offers C4 diagram kinds (C4Context, C4Container, C4Component, C4Dynamic, C4Deployment) with System_Boundary and Rel syntax [5]; the course accepts C4Container as an optional second input once flowcharts work.
- Diagrams are often incomplete or stale, so every input is treated as a claim with a source, not as ground truth (client input, safety section).

Mermaid flowchart grammar the parser must handle

- Header: 'flowchart LR' (or TB, TD, BT, RL); 'graph' is an accepted alias [10].
- Node shapes carry meaning in our convention: id[Label] for a service, id[(Label)] cylinder for a data store, id((Label)) for an external actor [10].

- Edges: A-->B, A-->|HTTPS|B and A-- text -->B carry protocol labels; A-.->B dotted edges are reserved for 'assumed' or 'planned' flows in our convention [10].
- subgraph id [title] ... end blocks map to trust boundaries; a lowercase 'end' inside a label breaks the flowchart, which is a real parsing failure to test [10].
- Anything the grammar does not cover (click handlers, style lines, comments with free text such as 'legacy batch') goes to the unparsed list with line numbers.

The JSON architecture schema

- Top level: components, data_stores, identities, external_systems, flows, boundaries, each an array of objects with a stable id.
- Use required for id, kind and source; remember a null does not satisfy required, so 'unknown' must be an explicit value, not null [11].
- Set additionalProperties: false so a typo such as 'protocl' fails validation instead of being silently ignored [11].
- Keep the schema flat (flows reference ids) rather than nested; Claude structured outputs do not support recursive schemas, numeric minimum/maximum or string length constraints, and require additionalProperties false [21].
- Markdown ADRs are the third input; an ADR records one decision and its rationale [7], and MADR gives a predictable section layout for parsing [13].

Treat every input as untrusted

- Architecture documents come from many authors; text inside an ADR can contain instructions aimed at the model, which is indirect prompt injection [20].
- Mitigations used in the build: mark document content as data in prompts, validate model output against the schema in code, and keep the model away from write tools [20].
- The reviewer's agent gets read-only tools only (read_file, list_inputs); open-ended tools such as arbitrary shell execution are excluded [25].
- Authorization is enforced by code, not by asking the model to behave; OWASP calls this complete mediation [25].

The AI-assisted build workflow

- Write a one-page product brief first, then ask the coding assistant for repository structure and interface contracts before code (client shared workflow).
- Put project conventions in ./CLAUDE.md (run /init for a starting file) and keep it short; CLAUDE.md is context, not enforcement [27].
- Add a PreToolUse hook in .claude/settings.json that blocks edits to evals/expected/ so the assistant cannot change the answer key; exit code 2 blocks the call [26].
- Build the deterministic baseline (parsers, schemas, tests) before any LLM call, then add one vertical slice at a time (client milestone M1).

Instructor demo.

1. Open the reference repository layout (ingestion/, system_model/, threat_model/, review_rules/, evidence/, evals/, fixtures/, tests/) and the product brief.
2. Show docs/payments.mmd rendered on GitHub inside a mermaid code fence, and run the 'info' block to show the Mermaid version GitHub uses [6].
3. Run the Mermaid parser on payments.mmd and show the parsed nodes, edges and subgraphs as JSON.
4. Introduce a deliberate break (a node labelled 'end of day batch') and show the parser reporting the failure with a line number instead of crashing [10].
5. Validate docs/payments.json against the schema, then add an undeclared key and show the additionalProperties: false rejection [11].
6. Show a CLAUDE.md for the project and a PreToolUse hook that refuses edits under evals/expected/, then ask Claude Code to 'fix the failing eval' and show the block [26][27].

Hands-on lab. Parse the reference architecture inputs and report what could not be parsed.

1. Clone the lab repository and run the setup command from the README; confirm pytest runs with zero tests failing.
2. Read fixtures/secure/payments.mmd, fixtures/secure/payments.json and fixtures/secure/adr/*.md.
3. Ask Claude Code to propose the parser interface (parse_mermaid(path) → ParseResult with elements, unparsed, warnings) and review it before any code is written.
4. Implement the Mermaid parser for flowchart headers, the three node shapes, labelled edges and subgraphs.
5. Implement JSON schema validation with additionalProperties: false and required ids.
6. Implement a Markdown ADR splitter that extracts title, status and sections by MADR heading names [13].
7. Write pytest cases for each fixture, including the flawed fixture that contains an unsupported click directive and a lowercase 'end' label.
8. Run the ingestion CLI on both fixture sets and save the unparsed report to out/ingest_report.json.

Lab success criteria.

- Every fixture file produces either parsed elements or an entry in the unparsed list; nothing is silently dropped.
- The schema rejects an undeclared key and a missing id with a clear message.
- Parser tests pass in one documented command.
- The ingest report lists at least the known unparsed note ('legacy batch') with its source line.

Deliverable. Ingestion layer.

Common pitfalls.

- Writing a regex that 'mostly works' for Mermaid and silently drops lines it does not understand.
- Allowing null for unknown values, which then passes 'required' checks inconsistently; use an explicit 'unknown' enum value.

- Starting with screenshot or image diagram parsing because it demos well; the client input explicitly defers it.
- Letting the coding assistant edit the expected eval results to make tests pass.
- Treating text inside ADRs as instructions rather than data.

Knowledge check.

1. Why does the course defer image-diagram extraction? Answer: Image parsing is not reliable enough to be trusted; the client input asks to start with Mermaid, JSON and Markdown and to make any later diagram parsing confidence-scored and reviewable.
2. What does additionalProperties: false buy you in the architecture schema? Answer: Any undeclared key, such as a misspelled field, makes the document invalid instead of being ignored [11].
3. A label in a Mermaid node reads 'end of day'. What happens and how should the parser respond? Answer: Lowercase 'end' can break a Mermaid flowchart [10]; the parser should record the failure with its line number in the unparsed list, not guess.
4. Where should the rule 'never edit expected eval results' live so it is enforced? Answer: In a PreToolUse hook, because CLAUDE.md is context and not enforced configuration [26][27].

Slides and assets to prepare.

- Diagram of the reviewer pipeline: inputs, ingestion, system graph, threat model, review rules, findings, architect decisions, cross-check, report.
- One-page cheat sheet of the supported Mermaid subset and the course's shape conventions.
- The JSON architecture schema file with annotated required fields.
- Two fixture sets (secure and flawed) with a README describing the planted defects, for instructors only.
- Sample .claude/settings.json with the eval-protection hook and a starter CLAUDE.md.

Sources for this module. 2, 4, 5, 6, 7, 10, 11, 13, 20, 21, 25, 26, 27

Module 2: The system graph with provenance

Goal. Turn parsed inputs into one typed graph where every node and edge carries its source, confidence and any ambiguity.

Outline topics.

- Components, data stores, identities, external systems, protocols, data classifications, trust boundaries
- Which source supports each edge; unknown and ambiguous relationships are first-class

Learning objectives.

- Learners can define node types for components, data stores, identities, external systems and trust boundaries, and edge attributes for protocol, data classification and authentication.
- Learners can attach provenance (file, line, input type) to every node and edge.

- Learners can represent unknown and ambiguous relationships as first-class graph elements.
- Learners can merge evidence from the diagram and ADRs without losing which source said what.

What goes in the graph

- OWASP's modeling phase lists the element types a DFD needs: external entities, processes, data stores, data flows and trust boundaries [2].
- pytm is a useful reference for element classes (Server, Datastore, Dataflow, Boundary, Actor, ExternalEntity, and newer LLM and Agent elements) and for boundary helpers such as crosses() and inside() [12].
- Course node kinds: component, data_store, identity, external_system; boundaries are separate objects that nodes belong to.
- Edge attributes: protocol (e.g. HTTPS, gRPC, AMQP, unknown), data_classification (public, internal, confidential, pii, unknown), authn (mtls, oauth2, api_key, none, unknown), direction.

Provenance on every element

- Every node and edge has a sources list: {input: 'payments.mmd', line: 14} or {input: 'ADR-012', section: 'Decision Outcome'}.
- Use a NetworkX MultiDiGraph so two different sources can each assert an edge between the same nodes as separate keyed edges [22].
- Store attributes in the edge's dictionary and update them with the key, e.g. G.edges[u, v, key]['confidence'] = 'low' [22].
- The promo's '27 edges from diagram, 4 from ADR-012' line is the kind of summary provenance makes possible; it is illustrative fixture output, not a benchmark.

Unknown and ambiguous are first-class

- An edge drawn without a label gets protocol 'unknown', not a default of HTTPS.
- An ambiguous edge is one where sources disagree (diagram says HTTPS, ADR says message queue) or a node name matches two components; both get flagged, not resolved silently.
- Ambiguities produce questions for the architect, which feed the findings module later.
- Diagram parsing should be confidence-scored and reviewable (client safety section).

Using the model for extraction, safely

- Claude can propose graph elements from prose ADRs, but the output must match a JSON schema; request it with output_config.format of type json_schema [21].
- Keep the schema flat (nodes array, edges array referencing ids) because recursive schemas are unsupported [21].
- Every model-proposed element must cite a source span; code rejects elements whose cited text is not found in the input.
- Model-proposed elements are marked origin: 'llm_extracted' so reviewers can filter them.

Instructor demo.

1. Load the parsed payments inputs and build the MultiDiGraph; print node and edge counts by kind.
2. Show one edge with two keyed entries, one from the diagram and one from an ADR, with different protocol values [22].
3. Run the ambiguity detector and show the two flagged edges and the generated architect questions.
4. Ask Claude to extract flows from ADR-012 prose with a json_schema output format, then show code rejecting one element whose source quote does not appear in the ADR [21].
5. Export the graph as JSON and render a Mermaid view with trust boundaries as subgraphs and unknown edges dotted.

Hands-on lab. Define the typed system graph schema and build the graph for the reference design.

1. Use Claude Code to draft a Pydantic model set for Node, Edge, Boundary and Source, with enums including 'unknown'.
2. Generate the JSON schema from the models and check it against the structured-output limits (no recursion, no numeric bounds) [21].
3. Write the builder that converts ParseResult objects into a MultiDiGraph with source lists on every element [22].
4. Add the ambiguity detector: conflicting attributes across sources, unlabeled edges, nodes outside any boundary.
5. Add an LLM extraction step for ADR prose that returns schema-valid elements with a quoted source span.
6. Write a check that rejects any LLM element whose quote is not in the source document.
7. Build graphs for both secure and flawed fixtures and save out/graph_secure.json and out/graph_flawed.json.
8. Render both as Mermaid and compare them by eye with the original diagrams.

Lab success criteria.

- 100 percent of nodes and edges have at least one source entry.
- Unknown protocol, classification or authentication values are explicit, never null or defaulted.
- The flawed design produces at least the planted ambiguous edges in the ambiguity list.
- LLM-extracted elements without a verifiable quote are rejected and logged.

Deliverable. Parsed architecture model.

Common pitfalls.

- Using a simple DiGraph, so a second source overwrites the first edge's attributes.
- Resolving conflicts automatically (latest source wins) instead of surfacing them.
- Defaulting missing protocols to HTTPS, which hides exactly the gaps a reviewer should find.
- Accepting model-extracted elements without checking the cited text exists.

Knowledge check.

1. Why a MultiDiGraph rather than a DiGraph? Answer: It keeps parallel edges between the same nodes as separate keyed entries, so each source's claim survives with its own attributes [22].
2. An edge in the diagram has no label. What protocol does the graph record? Answer: 'unknown', with the diagram as its source, and it becomes a candidate question for the architect.
3. Why must the extraction schema be flat? Answer: Claude structured outputs do not support recursive schemas, so nodes and edges are separate arrays linked by id [21].

Slides and assets to prepare.

- Graph schema reference card (node kinds, edge attributes, enum values including unknown).
- Before/after picture: the payments diagram and the rendered provenance graph with dotted unknown edges.
- Worked example of an ambiguous edge with the two source excerpts side by side.
- Starter Pydantic models file for learners who fall behind.

Sources for this module. 2, 12, 21, 22

Module 3: Requirements extraction

Goal. Extract security requirements and decisions from ADRs into a register with IDs, source references, verification status and links to graph nodes.

Outline topics.

- Security requirements and architecture decisions as explicit statements with IDs, source references and verification status

Learning objectives.

- Learners can define a requirement record with ID, statement, source, type, linked nodes and verification status.
- Learners can extract requirements from MADR-style ADRs and keep the exact source wording.
- Learners can map a requirement to an external standard identifier such as an ASVS requirement where useful.
- Learners can distinguish a stated requirement from an observed fact.

Where requirements hide in ADRs

- An ADR records one decision and its rationale; the project's set of ADRs is its decision log [7].
- MADR places the chosen option and justification under Decision Outcome, consequences under it, and how compliance is checked under Confirmation [13].
- Status matters: an ADR marked rejected or 'superseded by ADR-0123' should not produce active requirements [13].
- Requirements also appear as plain sentences ('PII must be encrypted at rest'), which is where Claude extraction helps.

The requirement record

GenNoor Tech TRAINING INNOVATE BUILD for more info - https://gennoor.com

- Fields: id (REQ-07), statement (verbatim), normalized_statement, source {adr: 'ADR-004', section, line}, category (encryption, authn, authz, logging, secrets, availability, privacy), linked_nodes, verification_status.
- verification_status values: unverified, design_supported (graph shows it), evidence_found (implementation evidence), contradicted, not_applicable.
- Every newly extracted requirement starts as 'unverified', as in the promo's 'REQ-07 PII encrypted at rest, status: unverified'.
- Where useful, map to a standard: ASVS 5.0.0 requirements should be cited with the version prefix, e.g. v5.0.0-1.2.5 [24].

Linking requirements to the graph

- A requirement about 'ledger data' must link to the ledger-db node; unmatched requirements are flagged, not dropped.
- Use exact id matches first, then aliases declared in the JSON schema, then a model suggestion that a person confirms.
- Requirements with no linked node are themselves findings: the design does not show where the requirement applies.
- Keep 'what the ADR says' separate from 'what the system does'; the register never upgrades status on the basis of a diagram label alone.

Extraction with Claude and deterministic checks

- Ask for a JSON array of requirement candidates with the verbatim source sentence, using a json_schema output format [21].
- Code checks that each verbatim sentence appears in the ADR and rejects any that does not, which catches paraphrased or invented requirements [19].
- Duplicates across ADRs are merged with both sources kept.
- A person reviews the register before it drives threat generation.

Instructor demo.

1. Open ADR-004 and ADR-009 from the payments fixture and highlight requirement-bearing sentences.
2. Run the extractor and show REQ-07 'PII encrypted at rest' created with status unverified and source ADR-004.
3. Show a superseded ADR being skipped and the log line explaining why [13].
4. Show one model candidate rejected because its quoted sentence is not in the ADR.
5. Link requirements to graph nodes and show the one requirement that matches no node.

Hands-on lab. Extract requirements from the ADRs and link them to graph nodes.

1. Define the Requirement Pydantic model and verification_status enum.
2. Implement a deterministic pass that picks sentences containing must, shall, required or never under Decision Outcome and Confirmation.

3. Add a Claude pass that proposes additional requirements with verbatim quotes, using structured output.
4. Reject candidates whose quote is not found verbatim; log each rejection.
5. Skip ADRs with status rejected, deprecated or superseded.
6. Link each requirement to graph nodes by id or alias, and flag unlinked ones.
7. Write the register to out/requirements.json and a Markdown table for reviewers.

Lab success criteria.

- Every requirement has an ID, a verbatim source sentence and a source reference.
- All requirements start as unverified.
- No requirement comes from a superseded ADR.
- Unlinked requirements appear in a separate list with a suggested question.

Deliverable. Requirement register.

Common pitfalls.

- Letting the model paraphrase requirements so the register no longer matches the ADR wording.
- Marking a requirement as met because the diagram has a label such as 'encrypted'.
- Ignoring ADR status and extracting requirements from rejected decisions.
- Dropping requirements that do not link to a node instead of flagging them.

Knowledge check.

1. A diagram node is labelled 'ledger-db (encrypted)'. What status does REQ-07 get? Answer: It can at most be design_supported; it stays short of evidence_found until implementation evidence is checked, and the label itself is recorded as a source.
2. How do you catch an invented requirement from the model? Answer: Require a verbatim quote and reject any candidate whose quote does not appear in the source ADR [19][21].
3. Which MADR status values should stop extraction? Answer: rejected, deprecated and 'superseded by ...' [13].

Slides and assets to prepare.

- Annotated MADR template showing where requirements usually appear.
- Requirement register example (Markdown table and JSON).
- Verification status ladder graphic: unverified, design_supported, evidence_found, contradicted.
- Optional ASVS mapping sheet for the categories used in the course.

Sources for this module. 7, 13, 19, 21, 24

Day 2

Module 4: Method-driven threat generation

Goal. Generate STRIDE threat candidates per modeled element, accept only those linked to real nodes or flows, and give each one assumptions, evidence and a validation question.

Outline topics.

- STRIDE or a defined threat library applied to components and data flows
- The LLM suggests candidates; every candidate must link to a real modeled component or flow
- Assumptions, evidence references and a validation question per threat

Learning objectives.

- Learners can state the six STRIDE categories and turn each into pointed questions for a component or flow.
- Learners can build a threat library that maps categories and conditions to element types.
- Learners can use Claude to propose candidates and reject any that do not reference a modeled element.
- Learners can write the assumption, evidence reference and validation question for each threat.

STRIDE as the method

- Spoofing (using another identity's credentials), Tampering (unauthorized modification of data at rest or in transit), Repudiation (denying an action without proof otherwise), Information Disclosure, Denial of Service and Elevation of Privilege [1].
- Microsoft uses STRIDE to formulate pointed questions such as 'How can an attacker change the authentication data?' [1].
- OWASP uses STRIDE for illustration and notes other methods exist; the course uses STRIDE because it is widely understood by architects [2].
- The Manifesto's second question, 'What can go wrong?', is what this module answers [3].

A threat library before any model call

- Write rules like pytm's: each threat has an ID, target element types, a condition over attributes and a description [12].
- Examples: tampering on a flow where authn is none or unknown and it crosses a boundary; information disclosure on a data store with classification pii and encryption unknown.
- Boundary-aware conditions matter; pytm offers helpers such as target.crosses(Boundary) and target.inside(Boundary) [12].
- Do not copy pytm's eval() approach for conditions; its own README warns threats.json strings run through eval() [12]. Use Python predicates instead.

Claude as a candidate generator

- Prompt with the graph JSON and the library, ask for candidates as structured output with fields threat_id, category, target_ref, assumption, evidence_refs, validation_question [21].

- Code validates that target_ref is an existing node id or edge key; anything else is rejected and logged, as the promo shows for 'SQL injection in reporting service (component not in model)'.
- This is the defense against confident but fabricated output that OWASP calls misinformation [19].
- Candidates are proposals; the deterministic library output and model output are merged and de-duplicated by (category, target).

Writing a good threat record

- Assumption: what must be true for the threat to apply, e.g. 'mTLS not enforced at ingress (ADR-009 silent)'.
- Evidence: edge or node ids and source lines that justify the candidate, e.g. edge#12 from payments.mmd.
- Validation question: what the architect must answer, e.g. 'Is mTLS enforced at the ingress?'
- Context for that example: in Istio, PERMISSIVE mode accepts plaintext or mTLS and only STRICT requires client certificates, so 'we use a mesh' does not settle the question [16].
- Response options later are mitigate, eliminate, transfer or accept [2].

Instructor demo.

1. Walk the payments graph and run the deterministic library; show threats produced per element.
2. Prompt Claude for extra candidates with the structured-output schema and show the raw response [21].
3. Run the validator and show one plausible candidate rejected because its target is not in the graph.
4. Open T-031 (Tampering, web → payments-api) and read its assumption, evidence and validation question aloud.
5. Show how the same flow in the secure fixture, which has a STRICT PeerAuthentication in the evidence, produces no T-031 [16].

Hands-on lab. Generate STRIDE candidates with the agent and reject every one that does not map to the graph.

1. Write at least eight library rules covering all six STRIDE categories.
2. Implement rule evaluation as Python predicates over node and edge attributes, including boundary crossing.
3. Define the ThreatCandidate schema and call Claude with the graph and library as context.
4. Implement the link validator and the rejection log.
5. Merge library and model candidates, de-duplicate and assign T-IDs.
6. Fill assumption, evidence_refs and validation_question for every accepted threat; reject threats missing any of the three.
7. Run on secure and flawed fixtures and save the threat registers.

Lab success criteria.

- Every accepted threat links to an existing node or edge.

- Every accepted threat has an assumption, at least one evidence reference and a validation question.
- The rejection log contains every model candidate that failed validation, with the reason.
- The flawed fixture produces threats for the planted unauthenticated cross-boundary flow.

Deliverable. Threat register with linked components.

Common pitfalls.

- Asking the model for 'all threats' without a graph, which produces generic lists that cannot be linked.
- Evaluating library conditions with eval() on strings.
- Treating a threat candidate as a finding; at this stage it is a hypothesis.
- Writing validation questions that cannot be answered yes or no.

Knowledge check.

1. Name the six STRIDE categories. Answer: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege [1].
2. The model proposes 'SQL injection in the reporting service', but no such component exists. What happens? Answer: The validator rejects it because its target is not in the graph, and logs the rejection.
3. Why is 'the services run in a mesh' not enough to close a tampering threat on a flow? Answer: Mesh mTLS can be PERMISSIVE, which accepts plaintext; only STRICT requires client certificates, so the evidence must show the mode [16].

Slides and assets to prepare.

- STRIDE card deck: one card per category with definition and two example questions.
- Threat library starter file with eight rules.
- Threat record template with a filled example (T-031).
- Rejection log example showing fabricated candidates.

Sources for this module. 1, 2, 3, 12, 16, 19, 21

Module 5: Architecture review rules

Goal. Implement deterministic review rules for authentication, authorization boundaries, secrets, logging, encryption, data minimization and failure handling, and run them on secure and flawed designs.

Outline topics.

- Missing or ambiguous authentication, authorization boundaries, secrets handling, logging, encryption, data minimization, failure handling
- Security, privacy and resilience review

Learning objectives.

GenOorTech INNOVATE. BUILD. For more info - https://genoor.com

- Learners can express each review area as a rule over graph attributes and requirements.
- Learners can distinguish 'missing' from 'ambiguous' in rule results.
- Learners can extend review rules to agentic components in a design.
- Learners can run the rule engine and compare results between secure and flawed fixtures.

Rule shape and results

- Each rule: id, area, applies_to, check function, result (pass, missing, ambiguous, not_applicable), evidence refs.
- Missing means the design states nothing; ambiguous means sources disagree or a value is unknown.
- Rules are deterministic, so the same graph always gives the same result, which makes regression testing possible.
- Rule results feed findings; they never become 'confirmed vulnerabilities' on their own.

Authentication, authorization and boundaries

- Every flow crossing a trust boundary needs an authn value other than none or unknown.
- Service-to-service authentication claims that rely on a mesh should name the mode; PERMISSIVE accepts plaintext [16].
- Kubernetes pods accept all ingress and egress unless a NetworkPolicy selects them, and a policy has no effect without a plugin that enforces it [23].
- Identity nodes must link to the components they can access; an identity with access to every data store is an authorization-boundary finding.

Secrets, encryption and data minimization

- Data stores with confidential or pii classification need an encryption-at-rest statement and a key owner.
- Do not infer encryption from platform: Kubernetes stores API data in etcd as plain text unless an encryption provider other than identity is first in the EncryptionConfiguration [15].
- AWS RDS storage_encrypted defaults to false in Terraform, so absence of a statement is a real gap [18].
- Data minimization: flows carrying pii to components with no stated need are flagged for a question.

Logging, failure handling and resilience

- Repudiation threats need audit logging of security-relevant actions on the component that performs them [1].
- Every external dependency should have a stated failure behavior (fail closed, queue, degrade); none stated is 'missing'.
- Single data stores without a backup or replica statement are flagged under resilience.
- Rate limiting or quota statements on internet-facing entry points address denial of service [1].

Rules for agentic components

- Many new designs include an LLM agent; pytm already models LLM and Agent elements [12].
- Rule: an agent node with write access to a data store or external system must show an approval step and least-privilege identity [25].
- Rule: an agent consuming external content (web, email, files) needs a stated prompt-injection control [20].
- Rule: authorization for agent actions must be enforced downstream, not by the model [25].

Instructor demo.

1. Show the rule registry and run all rules on the secure fixture: mostly pass, a few not_applicable.
2. Run on the flawed fixture and walk through three results: an unauthenticated cross-boundary flow, a pii store with no encryption statement, and an agent with write access and no approval step.
3. Show the difference between 'missing' (no statement) and 'ambiguous' (diagram says encrypted, ADR silent on keys).
4. Edit the flawed design to add the missing statement and rerun to show the result changes.
5. Show the pytest suite that pins rule outputs for both fixtures [28].

Hands-on lab. Implement the review rules and run them on secure and flawed reference designs.

1. Implement at least two rules for each of the seven review areas.
2. Add the three agentic-component rules.
3. Write a result model with pass, missing, ambiguous and not_applicable, plus evidence refs.
4. Run the engine on both fixtures and write out/rules_secure.json and out/rules_flawed.json.
5. Write parametrized pytest cases asserting expected results for selected rules on both fixtures [28].
6. Ask Claude Code to review the rules for false positives against the secure fixture, then inspect and accept or reject its suggestions by hand.

Lab success criteria.

- The secure fixture produces no 'missing' results for authn, encryption or secrets.
- The flawed fixture triggers every planted defect in the instructor key.
- Each result carries evidence refs to graph elements or requirements.
- Rule tests run in one command and pass.

Deliverable. Review rule engine.

Common pitfalls.

- Writing rules that look for keywords in labels instead of typed attributes.
- Assuming cloud or platform defaults are secure without evidence.
- Collapsing 'missing' and 'ambiguous' into one fail state, which loses the question to ask.
- Forgetting agentic components because they do not fit the classic DFD types.

Knowledge check.

1. A design shows a Kubernetes namespace and no NetworkPolicy. What is the default pod network behavior? Answer: Pods are non-isolated: all ingress and egress connections are allowed [23].
2. Why is 'Secrets are in Kubernetes' not an encryption statement? Answer: By default the API server stores resources in etcd as plain text; encryption needs a configured provider other than identity [15].
3. What rule applies to an agent that can write to a payments database? Answer: It must show an approval step, least-privilege identity and downstream authorization [25].

Slides and assets to prepare.

- Review area matrix: area, rule ids, attributes checked, typical questions.
- Planted-defect key for the flawed fixture (instructor only).
- Rule result JSON example with evidence refs.
- Short handout on platform defaults that commonly surprise reviewers (Kubernetes etcd, NetworkPolicy, RDS encryption flag).

Sources for this module. 1, 12, 15, 16, 18, 20, 23, 25, 28

Module 6: Evidence-graded findings

Goal. Combine threats and rule results into findings that keep observed facts, assumptions, missing information and recommended questions separate, and audit the report for unsupported claims.

Outline topics.

- Observed design facts, assumptions, missing information and recommended questions kept separate
- An unverified concern is never labelled a confirmed vulnerability
- Report schema

Learning objectives.

- Learners can define a finding schema with separate fields for observed, assumption, missing and question.
- Learners can assign an evidence grade and avoid calling any unverified concern a confirmed vulnerability.
- Learners can generate the narrative with Claude while code controls which facts appear.
- Learners can run an unsupported-claim audit on the generated report.

The finding schema

- Fields: id, title, linked_threats, linked_rules, linked_requirements, observed[], assumptions[], missing[], questions[], impact, recommendation, grade.
- Grades: design_fact (the design states it), inferred_risk (follows from facts plus an assumption), information_gap (cannot assess), never 'confirmed vulnerability' at design stage.

- The promo's F-12 shows the shape: observed 'REQ-07 exists (ADR-004)', assumption 'KMS-backed volume (not in diagram)', missing 'storage class, key policy'.
- The client input states it directly: do not label an unverified concern as a confirmed vulnerability.

Writing the narrative without inventing facts

- Claude drafts title, impact and recommendation from the structured finding; the observed and missing lists are filled by code only.
- Every sentence in the narrative that states a fact must cite an evidence id; sentences without one are flagged.
- This is a direct mitigation of misinformation and overreliance: automatic validation plus human review [19].
- State limitations in the report itself, as OWASP recommends communicating risks and limitations of generated content [19].

Unsupported-claim audit

- Parse the report, extract each factual claim, and check that its evidence id resolves to a graph element, requirement or source line.
- Unsupported-claim rate = claims without resolvable evidence / all claims (client evaluation metric).
- Evidence coverage = findings with a traceable source or modeled component / all findings (client metric).
- The promo's '100% findings linked to evidence' is this course's acceptance target, enforced by the schema; it is not an industry statistic.

Report format and recommendations

- Sections: scope and inputs, method, system summary, findings by grade, open questions, limitations, appendix of evidence.
- Recommendations follow the four responses: mitigate, eliminate, transfer or accept [2].
- Render a Mermaid diagram of the affected subgraph inside the Markdown report so it shows in GitHub [6].
- Use ASVS identifiers with version for recommendations that map to a verification requirement, e.g. v5.0.0-x.y.z [24].

Instructor demo.

1. Build F-12 from T-032, the encryption rule result and REQ-07, and show each field's source.
2. Ask Claude to write the narrative; then run the claim audit and show one sentence flagged ('the ledger uses AES-256') because no evidence supports it.
3. Fix the narrative by removing the claim and rerun the audit to zero unsupported claims.
4. Render the report in a GitHub pull request with the Mermaid subgraph [6].
5. Show the limitations section and discuss why it is mandatory.

Gemnoor Tech
For more info https://gemnoor.com

Hands-on lab. Produce the review report and audit it for unsupported claims.

1. Define the Finding model and grade enum with no 'confirmed' value.
2. Write the correlator that groups threats, rule results and requirements into findings by target element.
3. Fill observed, assumptions and missing from structured data in code.
4. Call Claude with structured output to draft title, impact, recommendation and questions.
5. Implement the claim audit and compute unsupported-claim rate and evidence coverage.
6. Render the report as Markdown with an embedded Mermaid subgraph per finding.
7. Run on both fixtures and fix any unsupported claims.

Lab success criteria.

- No finding uses the word 'confirmed' or a confirmed grade.
- Unsupported-claim rate is zero after fixes, and the audit output is saved.
- Every finding has at least one question or recommendation.
- The report has a limitations section listing input gaps.

Deliverable. Security architecture review report.

Common pitfalls.

- Letting the model write the observed facts section.
- Severity inflation: grading information gaps as high-severity vulnerabilities.
- Reports with no limitations section, which invite overreliance.
- Counting a citation to a whole document as evidence; it must point to an element or line.

Knowledge check.

1. What is the difference between an inferred risk and an information gap? Answer: An inferred risk follows from stated facts plus a named assumption; an information gap means the design does not contain enough to assess the concern at all.
2. How is unsupported-claim rate calculated? Answer: Claims without resolvable evidence divided by all factual claims in the report.
3. Which four responses can a recommendation take? Answer: Mitigate, eliminate, transfer or accept [2].

Slides and assets to prepare.

- Finding anatomy slide using F-12.
- Report template in Markdown.
- Claim audit output example.
- Grade definitions card for architects.

Sources for this module. 2, 6, 19, 24

Day 3

Module 7: Architect review and decision records

Goal. Let architects accept, reject, defer or request more evidence on each finding, capture rationale and owner, and track changes across design revisions.

Outline topics.

- Accept, reject, defer or request more evidence; rationale and owner captured
- Change tracking across design revisions

Learning objectives.

- Learners can implement a decision workflow with four outcomes, rationale, owner and due date.
- Learners can store decisions in a reviewable form in the repository.
- Learners can re-run the review on a revised design and carry decisions forward where findings still match.
- Learners can run a review session that answers validation questions.

The decision model

- Outcomes: accept (the finding is valid and will be addressed or risk accepted), reject (not valid, with reason), defer (valid, later), more_evidence (cannot decide yet).
- Required fields: decision, rationale, owner, decided_by, date, and due date for defer and more_evidence, as in the promo's 'MORE EVIDENCE owner: platform-lead due: 2026-11-05'.
- The Manifesto values people and collaboration over tools; the tool records the architect's call, it does not make it [3].
- Accepting risk is a legitimate response alongside mitigate, eliminate and transfer [2].

Decisions as records in Git

- Store decisions as YAML or JSON next to the design, one file per finding, reviewed in pull requests.
- Significant decisions become ADRs; MADR's Decision Outcome and Consequences sections fit them [13].
- When a decision changes an earlier ADR, mark the old one 'superseded by ADR-00NN' [13].
- GitHub renders Mermaid in the pull request so reviewers see the affected subgraph [6].

Change tracking across revisions

- Finding identity is a fingerprint of (category, target element id, rule id), not the text, so it survives wording changes.
- On re-run: matched findings keep their decision; new findings are 'new'; missing findings are 'resolved in revision N' with the diff that removed them.
- Graph diffs (added, removed, changed edges and attributes) are part of the review record.
- Continuous refinement over a single delivery is a Manifesto value; the workflow is built to be re-run [3].

GenMoor Tech TRAINING INNOVATE BUILD For more info - https://genmoor.com

Running the review session

- Group findings by owner and go through validation questions first; answers become new sources in the graph.
- An answer such as 'mTLS is STRICT mesh-wide' becomes a source that a later cross-check can test [16].
- Time-box: decisions not reached go to more_evidence with an owner.
- Close with the fourth Manifesto question: did we do a good enough job? [3]

Instructor demo.

1. Open the flawed design report and run a mock review with two learners playing architect and platform lead.
2. Record accept, reject, defer and more_evidence decisions through the CLI and show the generated YAML files.
3. Open a pull request with the decisions and show the rendered Mermaid subgraph [6].
4. Apply a design revision (add mTLS STRICT statement, remove a flow) and re-run the review.
5. Show the change report: decisions carried forward, one finding resolved, one new finding.

Hands-on lab. Run a review session and record decisions against the findings.

1. Implement the Decision model with validation (due date required for defer and more_evidence).
2. Implement the fingerprint function for finding identity.
3. Build the CLI: archreview decide F-12 --decision more_evidence --owner platform-lead --due 2026-11-05 --rationale '...'.
4. In pairs, review the flawed design report and record a decision for every finding.
5. Write one MADR ADR for the most significant accepted change [13].
6. Apply the provided revision patch to the design and re-run the review.
7. Generate the change report and commit decisions in a pull request.

Lab success criteria.

- Every finding has a decision with rationale and owner.
- Decisions survive a re-run on the revised design where the finding still exists.
- The change report lists new, carried-forward and resolved findings correctly.
- At least one ADR is written in MADR format with a status.

Deliverable. Decision workflow.

Common pitfalls.

- Using finding text as identity, so every rewording loses the decision.
- Accepting risk without an owner or rationale.
- Letting 'more evidence' items stay open forever without a due date.
- Allowing the tool to auto-decide based on severity.

Knowledge check.

1. What must a 'more_evidence' decision include? Answer: Rationale, owner and due date.
2. How does a decision survive a re-run? Answer: Findings are matched by a fingerprint of category, target element and rule, not by text.
3. When should an earlier ADR be marked superseded? Answer: When a new decision replaces it; MADR status 'superseded by ADR-0123' records the link [13].

Slides and assets to prepare.

- Decision outcome flowchart.
- Decision YAML example and MADR ADR example.
- Revision patch for the flawed design.
- Review session facilitation script with timing cues per finding group.

Sources for this module. 2, 3, 6, 13, 16

Module 8: Design-to-implementation comparison

Goal. Compare modeled requirements with Terraform, Kubernetes and repository evidence, and report coverage and gaps without claiming proof of conformance.

Outline topics.

- Comparing modeled requirements with Terraform, Kubernetes or repository evidence
- Reporting coverage and gaps, never claiming full proof of conformance

Learning objectives.

- Learners can produce and parse Terraform plan JSON safely as evidence.
- Learners can read Kubernetes manifests for security-relevant settings such as Pod Security labels, NetworkPolicy, mesh mTLS and encryption configuration.
- Learners can map a requirement to evidence checks and report covered, gap or contradicted.
- Learners can write a coverage report that states what was not checked.

Terraform as evidence

- Run terraform plan -out=tfplan then terraform show -json tfplan; the JSON has resource_changes with address, type and change.after [14].
- Saved plan files contain sensitive values in cleartext; treat them as sensitive artifacts and do not hand them to the model raw [8].
- Mask values marked in after_sensitive before any display or model call [14].
- Use -detailed-exitcode in CI: 0 no changes, 1 error, 2 changes present [8].

Encryption evidence for REQ-07

- aws_db_instance.storage_encrypted defaults to false; evidence must show true and a kms_key_id [18].

- aws_kms_key.enable_key_rotation is optional and must be set explicitly; if it is not in the configuration, rotation is 'unknown' or a gap, as in the promo's 'gap: key rotation unknown' [17].
- Record the file and line (infra/db.tf:22) as the evidence reference.
- A plan shows intended configuration, not the running account; say so in the report.

Kubernetes evidence

- Pod Security Standards levels: privileged, baseline, restricted [9]; check namespace labels against the requirement.
- NetworkPolicy objects show intended isolation, but only if the cluster's network plugin enforces them [23].
- Istio PeerAuthentication with mode STRICT in the root namespace is mesh-wide evidence for an mTLS requirement; PERMISSIVE is a gap [16].
- Secrets encryption at rest is set on kube-apiserver via --encryption-provider-config, usually outside application repositories; absence in the repo is 'not found', not 'failed' [15].

Reporting coverage honestly

- Each requirement gets covered (evidence found, with refs), gap (evidence not found where expected), contradicted (evidence shows the opposite) or out_of_scope.
- The promo line 'REQ-11 not found in evidence (reported as GAP, not failure)' is the rule.
- The report ends with 'conformance claimed: none beyond listed evidence'.
- OWASP's validation phase asks whether the model matches the real system; this module answers part of that with evidence, not all of it [2].

Instructor demo.

1. Run terraform plan -out and terraform show -json on the lab infra/ folder and show resource_changes for the database and key [8][14].
2. Show after_sensitive masking before the evidence goes into any prompt.
3. Run archreview crosscheck --tf infra/ --k8s deploy/ and show REQ-07 covered with a rotation gap [17][18].
4. Show the PeerAuthentication manifest in PERMISSIVE mode in the flawed fixture and the resulting contradiction for the mTLS requirement [16].
5. Open the coverage report and read the 'not checked' section.

Hands-on lab. Cross-check three requirements against infrastructure code and report the evidence gaps.

1. Run terraform init and terraform plan -out=tfplan in fixtures/flawed/infra with the provided dummy provider settings, then terraform show -json tfplan > plan.json.
2. Write a parser for plan.json that masks after_sensitive values.
3. Implement checks for REQ-07 (encryption at rest with KMS and rotation), REQ-09 (mTLS STRICT) and REQ-11 (network isolation via NetworkPolicy).

4. Parse deploy/*.yaml with a YAML loader and extract PeerAuthentication, NetworkPolicy and namespace labels.
5. Produce out/coverage.json with status and evidence refs per requirement.
6. Render the coverage report with a 'not checked' section and the no-conformance statement.
7. Repeat on the secure fixture and compare.

Lab success criteria.

- Three requirements each have a status and evidence references or a stated reason.
- No sensitive plan value appears in logs, prompts or the report.
- At least one gap is reported as a gap, not a failure.
- The report states that no conformance is claimed beyond listed evidence.

Deliverable. Conformance coverage report.

Common pitfalls.

- Treating absence of evidence in the repository as proof the control is missing in production.
- Passing raw plan files to the model and leaking secrets.
- Equating a NetworkPolicy object with enforced isolation without checking the network plugin.
- Reporting 'compliant' instead of 'covered by listed evidence'.

Knowledge check.

1. Why must a saved plan file be handled as sensitive? Answer: It stores sensitive values in cleartext even when terminal output hides them [8].
2. The Terraform has aws_kms_key with no enable_key_rotation. What do you report? Answer: Encryption key present, rotation not set in configuration: a gap with the file reference [17].
3. Is a PeerAuthentication in PERMISSIVE mode evidence for an mTLS requirement? Answer: No; PERMISSIVE accepts plaintext, so it is a contradiction or gap; STRICT is required [16].

Slides and assets to prepare.

- Evidence mapping table: requirement category, evidence source, check, typical gaps.
- Annotated plan.json excerpt showing resource_changes and after_sensitive.
- Kubernetes evidence cheat sheet (PSS labels, NetworkPolicy, PeerAuthentication, EncryptionConfiguration).
- Coverage report template with the no-conformance statement.

Sources for this module. 2, 8, 9, 14, 15, 16, 17, 18, 23

Module 9: Evaluation and capstone

Goal. Measure the reviewer against secure and intentionally flawed reference architectures, document its audit trail and limitations, and demo the end-to-end tool.

Outline topics.

- Secure and intentionally flawed reference architectures; threat coverage, unsupported claims, evidence linkage, usefulness
- Audit trail, limitations and capstone demo

Learning objectives.

- Learners can build a golden evaluation set of expected threats, findings and benign cases.
- Learners can compute threat coverage, unsupported-claim rate, evidence linkage and false positives.
- Learners can run evaluations before and after prompt or model changes.
- Learners can present the tool with its limitations and audit trail.

Golden sets and metrics

- Expected threats per fixture in evals/expected/, written by the instructor and protected from edits by the hook from day one [26].
- Threat coverage (recall) = expected threats found / expected threats; precision = found threats in expected set / all found (client metrics).
- Unsupported-claim rate and evidence linkage come from the day-two audit.
- Benign cases matter: the secure design should produce few findings; measuring only recall rewards noisy reviewers.

Test harness

- Use @pytest.mark.parametrize over fixtures and metrics so each design and threshold is its own test [28].
- Mark known model weaknesses with pytest.param(..., marks=pytest.mark.xfail) so they are tracked, not hidden [28].
- Record the prompt version, model setting and library version with each run, since the client principles require logging tool and model versions.
- The promo's 'threat coverage 0.89, unsupported claims 0.02' line is an illustrative run, not a promised result.

Audit trail and limitations

- Log inputs read, graph version, rules run, model calls with prompt hash, rejections, decisions and evidence checked.
- Do not log secrets or sensitive plan values (client principles).
- Limitations to state: diagram-only inputs, stale designs, evidence limited to repositories provided, no runtime verification.
- Answer the Manifesto's last question for the tool itself: did we do a good enough job? [3]

Capstone acceptance

- Graph preserves provenance and marks ambiguous edges.
- Threats link to modeled elements and state assumptions.
- Reports separate facts from questions and hypotheses.

- Architects can record decisions; evaluation covers secure and flawed designs; any implementation comparison reports gaps and implies no full assurance (client acceptance criteria).

Instructor demo.

1. Run the full eval suite on both fixtures and show the metric table.
2. Change one prompt instruction and rerun to show a regression in unsupported-claim rate.
3. Revert and show the metrics restored, with both runs logged with versions.
4. Walk the audit trail for one finding from input line to decision.
5. Show a sample capstone limitations slide.

Hands-on lab. Capstone build and demo.

1. Choose a third architecture (provided options: internal HR portal, event-driven order system, or the learner's own sanitized design).
2. Write Mermaid or JSON inputs and two ADRs for it.
3. Create expected threats and benign expectations with a partner who did not write the design.
4. Run the full pipeline: ingest, graph, requirements, threats, rules, findings, decisions, optional cross-check.
5. Run the eval suite on all three designs and record metrics.
6. Prepare a ten-minute demo covering one finding end to end, metrics and limitations.

Lab success criteria.

- End-to-end run completes without manual code edits between steps.
- Metrics reported for secure, flawed and the new design.
- Zero unsupported claims in the final report.
- Limitations and audit trail presented.

Deliverable. End-to-end architecture reviewer.

Common pitfalls.

- Writing expected results after seeing tool output.
- Reporting only recall.
- Demos that skip the limitations slide.
- Using real company designs without sanitizing names, addresses and secrets.

Knowledge check.

1. Why include a secure reference design in the evaluation? Answer: To measure false positives and unsupported claims; a reviewer that flags everything would score high recall on the flawed design alone.
2. How do you track a known weakness without hiding it? Answer: Mark that case as xfail with pytest.param so it is visible in every run [28].
3. What must the audit trail never contain? Answer: Secrets and sensitive plan values.

Gennoor Tech
INNOVATE. BUILD.
For more info - https://gennoor.com

Slides and assets to prepare.

- Metric definitions slide.
- Capstone rubric (functionality, safety controls, testing and evaluation, engineering quality, evidence and explainability, documentation).
- Third-design option packs.
- Demo script template.

Sources for this module. 3, 26, 28

6. Capstone plan

A tool that ingests structured architecture inputs, builds a provenance-aware system graph, generates a linked threat model, produces evidence-graded findings and records architect decisions, with optional implementation cross-checks that report gaps rather than proof.

Build and demo an end-to-end architecture reviewer that ingests structured inputs, builds a provenance-aware graph, generates a linked threat model, produces evidence-graded findings, records architect decisions and, optionally, cross-checks implementation evidence while reporting gaps rather than proof. Learners run it on the secure, flawed and a third design.

Milestones

1. M1: deterministic baseline (parsers, schemas, graph, rules, tests) passes without any model call.
2. M2: model-assisted extraction and threat candidates with validation and rejection logs.
3. M3: findings report with zero unsupported claims and a limitations section.
4. M4: decision workflow with change tracking across one revision.
5. M5: implementation cross-check on at least three requirements.
6. M6: evaluation metrics on three designs and a ten-minute demo.

Evaluation notes

- Weighting follows the client rubric: working functionality 25%, security and safety controls 20%, testing and evaluation 20%, engineering quality 15%, evidence and explainability 10%, documentation and presentation 10%.
- Automatic fail on any finding graded or worded as a confirmed vulnerability, or any claim of full conformance.
- Reviewers check one finding end to end from input line to decision during the demo.
- Metrics must include false positives on the secure design, not only coverage on the flawed one.

Assessment weights

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. The 'diagram said encrypted, implementation said otherwise' opening is a teaching scenario, not a reported incident. It is realistic because common defaults diverge from what diagrams imply: aws_db_instance.storage_encrypted defaults to false [18], Kubernetes stores API data in etcd as plain text unless encryption is configured [15], and Istio's PERMISSIVE mode accepts plaintext connections [16]. Microsoft frames threat modeling as early design work [1], which is why a review done months before implementation can miss later drift.

GenNoorTech TRAINING INNOVATION LLC For more info - https://gennoor.com

Claim or on-screen text	Basis
6 STRIDE categories: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.	Matches Microsoft's STRIDE table [1].
'Applied only to components that really exist in the model.'	Course design rule from the client input (every candidate must link to a modeled component or flow); enforced by the validator in the threat-generation lab. It mitigates the misinformation risk OWASP describes [19].
'STRIDE categories, every flow.'	Teaching simplification. The course runs all six categories as prompts over every element and flow, but the library keeps only candidates whose conditions apply, so not every category yields a threat for every flow.
Constrained inputs: Mermaid, JSON schema, Markdown ADRs.	From the client build sequence; Mermaid flowchart syntax [10], JSON Schema object validation [11] and ADRs [7].
Terminal output: 'components: 14 … edges: 31 (27 from diagram, 4 from ADR-012) ambiguous: 2 … REQ-07 … unverified … unparsed: 1 note (legacy batch)'.	Illustrative output from the course's payments fixture, designed to match the flawed fixture contents; not measured data.
T-031 'assumes mTLS not enforced' with validation question 'Is mTLS enforced at the ingress?'	Scenario from the fixtures; realistic because mesh mTLS can be PERMISSIVE rather than STRICT [16].
'rejected: SQL injection in reporting service (component not in model)'.	Lab outcome of the link validator; a planted model candidate in the instructor demo.
Finding F-12: 'Ledger encryption at rest is a requirement, not an observed fact'; decision MORE EVIDENCE with owner and due date.	Illustrates the client's evidence-grading and decision rules; the due date is fictional fixture data.
'REQ-07 covered: aws_kms_key + encrypted volume (infra/db.tf:22) gap: key rotation unknown'.	Illustrative cross-check output; rotation must be set explicitly via enable_key_rotation [17], and storage encryption via storage_encrypted [18].
'REQ-11 not found in evidence (reported as GAP, not failure)' and 'conformance claimed: NONE beyond listed evidence'.	Client requirement to report coverage and gaps, never proof of conformance.
'eval: threat coverage 0.89 unsupported claims 0.02 evidence linkage 1.00'.	Illustrative numbers; the metric names come from the client's evaluation metrics table. Not a promised result.
Stats: 3 days hands-on; 2 reference designs, secure and flawed; 100% findings linked to evidence.	Course format and acceptance criteria. '100% linked' is enforced by the finding schema and audit in the labs; it is a design target, not an industry statistic.
'Review the architecture, not the slide.'	Tagline; consistent with the Manifesto value of finding and fixing design issues over checkbox compliance [3].

References

[1] Threats - Microsoft Threat Modeling Tool - Azure | Microsoft Learn. Microsoft Learn. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
Supports: Method-driven threat generation; promo STRIDE claim. Microsoft's Threat Modeling Tool, a core element of the SDL, uses the STRIDE model of six categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to help architects find and mitigate issues early, when they are relatively easy and cost-effective to resolve, and to turn vague worries into pointed questions such as 'How can an attacker change the authentication data?' and 'What happens if access is denied to the user profile database?'

[2] Threat Modeling - OWASP Cheat Sheet Series. OWASP. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
Supports: Course method; modeling, threat identification, response, validation. OWASP structures threat modeling around the Manifesto's four questions as phases: model the system (data flow diagrams showing trust boundaries, data flows, data stores, processes and external entities), identify and rank threats (STRIDE used 'for illustration purposes'), choose a response per threat (mitigate, eliminate, transfer or accept), then review and validate the model and mitigations.

[3] Threat Modeling Manifesto. Threat Modeling Manifesto working group. https://www.threatmodelingmanifesto.org/
Supports: Why this course; architect review values. The Manifesto's four questions are 'What are we working on?', 'What can go wrong?', 'What are we going to do about it?' and 'Did we do a good enough job?'; it values finding and fixing design issues over checkbox compliance, people and collaboration over processes and tools, and continuous refinement over a single delivery.

[4] Home | C4 model. C4 model (Simon Brown). https://c4model.com/
Supports: Architecture inputs: abstraction levels. The C4 model is a set of hierarchical abstractions (software systems, containers, components and code) with a matching set of hierarchical diagrams: system context, container, component and code diagrams.

[5] C4 Diagrams | Mermaid. Mermaid. https://mermaid.js.org/syntax/c4.html
Supports: Architecture inputs: C4 in Mermaid. Mermaid supports five C4 diagram kinds (C4Context, C4Container, C4Component, C4Dynamic, C4Deployment) with C4-PlantUML-compatible syntax such as System_Boundary(b2, "BankBoundary2") { ... } and Rel(SystemAA, SystemC, "Sends e-mails", "SMTP"); some features such as sprites, tags, links and legends are not implemented.

[6] Creating diagrams - GitHub Docs. GitHub Docs. https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams
Supports: Architect review: rendering diagrams in pull requests. GitHub renders Mermaid written inside a fenced code block tagged with the mermaid language identifier, and documents placing 'info' inside a mermaid block to check which Mermaid version GitHub is running.

[7] Architectural Decision Records (ADRs). adr.github.io. https://adr.github.io/
Supports: Requirements extraction; decision records. An Architectural Decision Record captures a single architectural decision and its rationale, and the collection of ADRs created and maintained in a project constitutes its decision log.

[8] terraform plan command reference | HashiCorp Developer. HashiCorp Developer. https://developer.hashicorp.com/terraform/cli/commands/plan
Supports: Design-to-implementation comparison. terraform plan previews changes without applying them; -out saves a plan file that stores sensitive values in cleartext and must be treated as a potentially sensitive artifact; -json emits machine-readable output and implies -input=false; -detailed-exitcode returns 0 for no changes, 1 for error and 2 for a non-empty diff.

[9] Pod Security Standards | Kubernetes. Kubernetes. https://kubernetes.io/docs/concepts/security/pod-security-standards/
Supports: Design-to-implementation comparison: Kubernetes evidence. Kubernetes defines three cumulative Pod

Security Standards: Privileged (entirely unrestricted, for trusted infrastructure workloads), Baseline (prevents known privilege escalations such as host namespaces, privileged containers and hostPath volumes) and Restricted (follows current pod hardening best practices).

[10] Flowcharts Syntax | Mermaid. Mermaid. https://mermaid.js.org/syntax/flowchart.html
Supports: Architecture inputs: Mermaid flowchart grammar. A Mermaid flowchart starts with 'flowchart' plus a direction (TB, TD, BT, RL, LR); node shapes are set by brackets, e.g. id1[(Database)] for a cylinder; links include A-->B, A-->|text|B, dotted A-.->B and thick A==>B; groups use 'subgraph id [title] ... end'; a lowercase word 'end' inside a node breaks the flowchart.

[11] Object - Understanding JSON Schema. JSON Schema. https://json-schema.org/understanding-json-schema/reference/object
Supports: Architecture inputs and system graph schema. In JSON Schema, properties listed under 'properties' are optional unless named in 'required'; a null value does not satisfy a required property; additionalProperties: false makes any undeclared key invalid; and additionalProperties only recognizes properties declared in the same subschema, so it can break allOf extension.

[12] OWASP pytm - A Pythonic framework for threat modeling. OWASP (GitHub). https://github.com/OWASP/pytm
Supports: System graph and threat library design reference. pytm models a system in Python with elements such as TM, Server, ExternalEntity, Datastore, Actor, Process, Dataflow, Boundary, Lambda, LLM and Agent; it outputs a DFD (--dfd), a sequence diagram (--seq), a report (--report) or JSON (--json); threats are JSON entries with SID, target, description, severity, condition, mitigations and references, and conditions can use helpers such as target.crosses(Boundary). Its threats.json strings run through eval().

[13] MADR - Markdown Architectural Decision Records. adr.github.io (MADR). https://adr.github.io/madr/
Supports: Requirements extraction; architect decision records. MADR 4.0.0 (released 2024-09-17) defines sections Context and Problem Statement, Decision Drivers, Considered Options, Decision Outcome (with Consequences), Confirmation, Pros and Cons of the Options and More Information; optional front matter status values include proposed, rejected, accepted, deprecated and 'superseded by ADR-0123'; files are named NNNN-title-with-dashes.md, typically in docs/decisions.

[14] JSON Output Format | Terraform | HashiCorp Developer. HashiCorp Developer. https://developer.hashicorp.com/terraform/internals/json-format
Supports: Design-to-implementation comparison: parsing plan JSON. terraform show -json <FILE> renders a plan or state as JSON with format_version, prior_state, configuration, planned_values and resource_changes; each resource_changes entry has address, type and a change object with actions (e.g. ["create"], ["update"], ["no-op"]), before, after, after_unknown and before_sensitive/after_sensitive, which should be combined with before/after to avoid displaying sensitive values.

[15] Encrypting Confidential Data at Rest | Kubernetes. Kubernetes. https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/
Supports: Review rules and conformance: encryption at rest. By default the API server stores plain-text representations of resources in etcd with no at-rest encryption; kube-apiserver --encryption-provider-config points to an EncryptionConfiguration (apiVersion apiserver.config.k8s.io/v1) whose first provider encrypts writes; if the first provider is 'identity' there is no confidentiality protection, and the check does not prove earlier data was migrated.

[16] PeerAuthentication | Istio. Istio. https://istio.io/latest/docs/reference/config/security/peer_authentication/
Supports: Threat generation and conformance: mTLS evidence. Istio PeerAuthentication defines mTLS requirements for incoming connections; modes are UNSET (inherit, otherwise PERMISSIVE), DISABLE, PERMISSIVE (plaintext or mTLS accepted) and STRICT (TLS with client certificate required); a mesh-wide policy lives in the root namespace.

[17] aws_kms_key resource documentation (terraform-provider-aws). HashiCorp (terraform-provider-aws on GitHub). https://raw.githubusercontent.com/hashicorp/terraform-provider-aws/main/website/docs/r/kms_key.html.markdown
Supports: Design-to-implementation comparison; promo cross-check panel. aws_kms_key's enable_key_rotation

is optional and must be enabled if rotation_period_in_days (90 to 2560) is set; deletion_window_in_days is 7 to 30 and defaults to 30; key_usage defaults to ENCRYPT_DECRYPT; policy takes a key policy JSON document.

[18] aws_db_instance resource documentation (terraform-provider-aws). HashiCorp (terraform-provider-aws on GitHub). https://raw.githubusercontent.com/hashicorp/terraform-provider-aws/main/website/docs/r/db_instance.html.markdown
Supports: Design-to-implementation comparison; promo hook. aws_db_instance's storage_encrypted defaults to false, kms_key_id takes the KMS key ARN, publicly_accessible defaults to false, iam_database_authentication_enabled maps IAM accounts to database accounts, and manage_master_user_password lets RDS manage the master password in Secrets Manager (not combinable with password).

[19] LLM09:2025 Misinformation - OWASP Top 10 for LLM Applications. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm092025-misinformation/
Supports: Method-driven threat generation; evidence-graded findings. OWASP describes misinformation as LLMs producing false or misleading information that appears credible, with hallucination as a major cause and overreliance as users failing to verify output; mitigations include grounding in trusted sources, cross-verification and human oversight, automatic validation of key outputs, and clear communication of limitations.

[20] LLM01:2025 Prompt Injection - OWASP Top 10 for LLM Applications. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
Supports: Architecture inputs: treating documents as untrusted. Indirect prompt injection occurs when an LLM accepts input from external sources such as websites or files; mitigations include constraining model behavior, defining and validating expected output formats with deterministic code, least privilege, human approval for high-risk actions, segregating and marking external content, and adversarial testing.

[21] Structured outputs - Claude API Docs. Anthropic. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: System graph schema; threat candidate output. Claude returns schema-valid JSON when output_config.format has type json_schema and a schema; strict: true on a tool validates tool inputs; additionalProperties must be false for objects; recursive schemas, external $ref, numeric constraints (minimum, maximum), string length constraints and array constraints beyond minItems 0 or 1 are not supported; the Python SDK's client.messages.parse() accepts a Pydantic model and returns parsed_output.

[22] MultiDiGraph - NetworkX documentation. NetworkX. https://networkx.org/documentation/stable/reference/classes/multidigraph.html
Supports: System graph implementation. NetworkX MultiDiGraph is a directed graph that stores multiple parallel edges between the same nodes, each identified by a key (by default the lowest unused integer); graphs, nodes and edges each hold attribute dictionaries, and for multigraphs the edge key is required to update an edge attribute: MG.edges[u, v, key][name] = value.

[23] Network Policies | Kubernetes. Kubernetes. https://kubernetes.io/docs/concepts/services-networking/network-policies/
Supports: Review rules and conformance: network isolation evidence. By default a pod is non-isolated for ingress and egress (all connections allowed); a pod becomes isolated for a direction only when a NetworkPolicy selects it with that direction in policyTypes; NetworkPolicy is implemented by the network plugin, and creating one without a supporting controller has no effect.

[24] OWASP Application Security Verification Standard. OWASP. https://asvs.owasp.org/
Supports: Requirements extraction: requirement identifiers. ASVS is a framework of security requirements for designing, developing and testing web applications and services; version 5.0.0 was released on 30 May 2025; requirements are numbered chapter.section.requirement and OWASP recommends citing them with the version, e.g. v5.0.0-1.2.5.

[25] LLM06:2025 Excessive Agency - OWASP Top 10 for LLM Applications. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
Supports: Architecture inputs: agent tool design; review rules for agentic systems. Excessive Agency arises from excessive functionality, permissions or autonomy; OWASP recommends minimizing extensions and their

functionality, avoiding open-ended tools such as arbitrary shell execution, minimizing permissions (e.g. read-only), requiring user approval for high-impact actions, and complete mediation: authorization implemented in downstream systems rather than relying on the LLM.

[26] Hooks reference - Claude Code Docs. Anthropic (Claude Code documentation).
https://code.claude.com/docs/en/hooks
Supports: Architecture inputs: AI-assisted workflow guardrails. Claude Code hooks are configured in ~/.claude/settings.json, .claude/settings.json or .claude/settings.local.json; a PreToolUse hook with a matcher such as "Bash" or "Edit|Write" can block a call by exiting with code 2 (stderr becomes the reason) or by returning hookSpecificOutput.permissionDecision "deny".

[27] How Claude remembers your project - Claude Code Docs. Anthropic (Claude Code documentation).
https://code.claude.com/docs/en/memory
Supports: Architecture inputs: project conventions for the AI-assisted build. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md, personal ones in ./CLAUDE.local.md and ~/.claude/CLAUDE.md; /init generates a starting CLAUDE.md; files can import others with @path syntax; .claude/rules/ holds topic files; CLAUDE.md is context, not enforced configuration, so a PreToolUse hook is needed to block an action.

[28] How to parametrize fixtures and test functions - pytest documentation. pytest.
https://docs.pytest.org/en/stable/how-to/parametrize.html
Supports: Evaluation and capstone: golden-set tests. @pytest.mark.parametrize("test_input,expected", [...]) runs a test once per parameter tuple, stacked decorators produce every combination, and pytest.param(..., marks=pytest.mark.xfail) marks an individual case as expected to fail.

For more info - https://gennoor.com

Gennoor Tech TRAIN. INNOVATE. LEAD.
For more info - https://gennoor.com